
Frameworks

Os próprios padrões talvez não sejam suficientes para desenvolver um projeto completo.

Em alguns casos pode ser necessário fornecer uma infraestrutura mínima específica para implementação, chamada framework.

Podemos selecionar uma “mini-arquitetura reutilizável que fornece a estrutura genérica e o comportamento para uma família de abstrações de software [...]”.

Gamma [1] e seus colegas descrevem as diferenças entre padrões de projeto e frameworks de uso de padrões da seguinte maneira:

1. Os padrões de projeto são mais abstratos do que os frameworks. Os frameworks podem ser incorporados ao código.
2. Os frameworks podem conter vários padrões de projeto, mas a recíproca jamais é verdadeira.
3. Os padrões de projeto são menos especializados do que os frameworks. Os frameworks apresentam um domínio de aplicação particular. Os padrões de projeto podem ser usados em praticamente qualquer tipo de aplicação.

A reutilização de implementações por meio de frameworks é fundamentalmente diferente da reutilização de implementações por meio de **componentes**.

Quando reutilizamos um framework, normalmente precisamos conhecer um certo número de diferentes classes que devem trabalhar em conjunto para que uma determinada funcionalidade seja reutilizada.

Quando reutilizamos componentes, ao contrário, basta criar o(s) componente(s) e acessá-lo(s) diretamente por meio de sua(s) interface(s).

Descrição de padrões

O projeto baseado em padrões pode ser pensado em 3 tarefas:

1. Reconhecimento de padrões na aplicação que se pretende construir;
2. Pesquisa para determinar se outros trataram do padrão;
3. Aplicação de um padrão apropriado para o problema em questão.

Como encontrar padrões que atendam às nossas necessidades?

Um dos principais problemas técnicos em projeto baseado em padrões é a incapacidade de encontrar padrões existentes quando há centenas ou milhares de candidatos.

A busca pelo padrão “correto” é tremendamente facilitada por um nome de padrão significativo.

Embora tenham sido propostos vários modelos de padrão distintos, quase todos contêm um subconjunto principal do conteúdo sugerido por Gamma [1] e seus colegas.

Erros comuns de projeto

Erros comuns de projeto

Uma série de erros comuns ocorre quando se usa projeto baseado em padrões.

Em alguns casos, não se dedicou tempo suficiente para entender o problema subjacente e seu contexto e forças e, como consequência, escolhe-se um padrão que parece correto, mas que, na verdade, é inadequado para a solução exigida.

Assim que o padrão incorreto é escolhido, recusamo-nos a admitir o erro e forçamos a adequação do padrão.

Algumas vezes um padrão é aplicado de forma demasiadamente literal, e as adaptações necessárias para o espaço do problema não são implementadas.

Os antipadrões descrevem soluções comuns a problemas de projeto que, em geral, têm efeitos negativos na qualidade do software.

Os antipadrões podem ser valiosos em orientar os desenvolvedores enquanto eles buscam maneiras de refatorar artefatos de software para melhorar a sua qualidade.

Uma ampla variedade de antipadrões de projeto foi identificada para ajudar os desenvolvedores a tomar decisões de refatoração.

Big ball of mud (grande bola de lama). Um sistema sem estrutura reconhecível.

Stovepipe system (sistema chaminé). Um conjunto de componentes mal relacionados e difícil de manter.

Boat anchor (âncora do barco). Manter parte de um sistema que não tem mais utilidade.

Lava flow (fluxo de lava). Manter código indesejável (redundante ou de baixa qualidade) porque removê-lo seria caro demais ou teria consequências imprevisíveis.

Spaghetti code (código espaguete). Programa cuja estrutura é quase incompreensível, especialmente devido ao mau uso das estruturas de código.

Copy and paste programming (programação por copiar e colar). Copiar código existente várias vezes em vez de criar soluções genéricas.

Silver bullet (bala de prata). Pressupor que uma solução técnica favorita sempre resolverá um processo ou problema maior.

Programming by permutation (programação por permutação). Tentar abordar uma solução pela modi cação sucessiva do código para ver se funciona.

<https://sourcemaking.com/antipatterns>

Referências

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides.
Padrões de Projetos: Soluções Reutilizáveis de Software Orientados a Objetos, volume 1.
Bookman, 1. edition, 2000.
- [2] R. S. Pressman and B. R. Maxim.
Engenharia de Software uma abordagem profissional, volume 1.
AMGH Editora Ltda, 9. edition, 2021.
- [3] S. R. Schach.
Engenharia de Software: Os paradigmas clássicos & orientados a objetos, volume 1.
McGraw-Hill, 7. edition, 2008.
- [4] I. Sommerville.
Engenharia de Software, volume 1.
São Paulo: Addison-Wesley, 10. edition, 2019.

- [5] M. T. Valente.
Engenharia de software moderna, volume 1.
Independente, 2020.
- [6] R. S. Wazlawick.
Engenharia de Software - Conceitos e Prática, volume 1.
Rio de Janeiro: GEN LTC, 2. edition, 2019.

Perguntas?

Obrigado pela atenção!